Data Streaming - Apache Kafka

This lesson explores the world of message brokers through Apache Kafka, a cornerstone of modern streaming architectures. It provides insights into real-time data processing and analytics by guiding students through designing and implementing robust streaming solutions with Apache Kafka. Additionally, students will explore Amazon MSK (Managed Streaming for Apache Kafka) to understand how it simplifies the deployment and management of Kafka applications.

Learning Objectives

- Fully explain a message broker service.
- Have a complete understanding of Apache Kafka and the design decisions made in its implementation.
- Know which data schemas and data types are used in streaming architectures.

Overview



Figure 1: Kafka logo. Source.

Apache Kafka is a modern message broker service that is popularly utilized within the backends of globally recognized platforms, including Netflix, LinkedIn, Uber, Twitter, and Spotify, to name a few.



Figure 2: Some companies that currently utilise Kafka within their tech stacks. Source

At its core, Kafka is based on a message log to which users can subscribe and/or publish data. The log is ordered as the events stream in and forms an append-only series of events that consumers can extract data from at any point in time.

Kafka combines three capabilities to enable an end-to-end streaming solution: publishing and subscribing to event streams, storing streams, and processing streams. Here we will first focus on the first and primary capability – Kafka's ability to function as a message broker.

Servers

Kafka is fundamentally a distributed system consisting of servers and clients that communicate via a high-performance TCP network protocol.

Kafka is run as a cluster of one or more servers that can span multiple data centers or cloud regions. Some of the servers function as the storage layer of the framework and are named **brokers**.
Other servers run Kafka Connect, a tool to continuously import and export data as event

streams to integrate Kafka with other systems, like relational databases or other Kafka clusters. Kafka servers are highly scalable and fault-tolerant.

Clients

Clients allow us to write distributed applications and microservices that read, write, and process streams of events in parallel. These higher-level abstractions and programs ship with the Kafka application for service-specific use cases.

Kafka Concepts

The figure below gives an overview of the Kafka system and producers writing to topics within the Kafka cluster, which, in turn, are partitioned based on message hashes. These messages are then consumed by consumers. We explain several Kafka-related concepts in greater detail below:

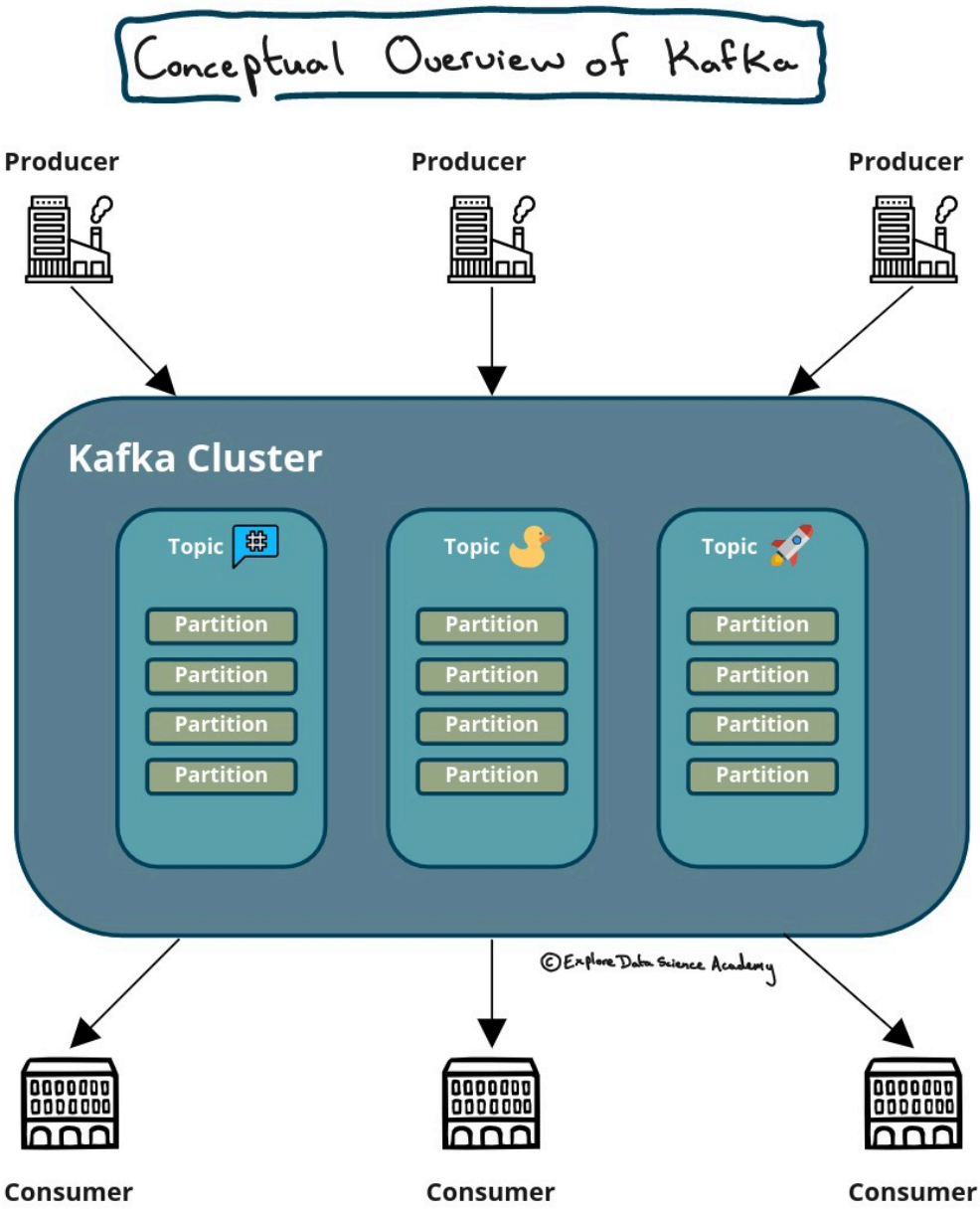


Figure 3: A high-level view of Kafka components.

Events

As already mentioned throughout our study of streaming services, events are records of a process or occurrence in the real world and are generally also referred to as messages within the streaming domain. In Kafka, an event includes a **key**, **value**, **timestamp**, and other optional metadata headers.

Producers

Producers are applications that write to Kafka. Producers send data directly to the broker that is the **leader** of the partition (more on leaders and followers later), without any routing. To assist the producer in finding the correct partition, each node in Kafka can answer a request for metadata about which servers are alive and where leaders for specific topics are.
The client will determine which partition to write to based on the partitioning rules set out (discussed below).

> **Designed to optimize throughput:**
> One way to optimize write speeds is by minimising I/O operations via batching messages. The Kafka producer will try to accumulate data in memory and send out larger batches in a single request. To balance this write efficiency with timeliness requirements, batching is only carried out until a set message count or latency threshold is exceeded. This behavior is completely configurable and allows for the balance of message latency and throughput.

Consumers

Consumers subscribe to events published to Kafka (i.e. they read from the streams generated by producers). Consumers and producers are decoupled, meaning that a producer never waits for a consumer or the other way around. Within Kafka, each consumer is responsible for keeping track of their own activity, which greatly reduces the overhead placed on the producer.

Instead of the broker **pushing** data to a consumer, Kafka is implemented with the consumer **pulling** data from the broker. This means that the consumer controls the rate of flow for incoming data and, thus, never falls behind while consuming data at its maximum possible rate. Here, if the consumer is pulling data at a rate slower than that which is pushed by the producer, it will simply catch up once the rate of production slows down or consumption capacity is scaled up. A further advantage of using a pull-based system is that a consumer can optimally decide on the scale of message batching used during data retrieval – on each pull it can decide to collect all or only a subset of available messages after its current position in the log.

Keeping Track of Log State

One of the key challenges in streaming is keeping track of which messages have been retrieved by consumers. If the broker records that a message has been consumed as soon as it is sent, the message can still be lost if the system fails before successful delivery to the consumer. Streaming applications try to mitigate this problem by adding an acknowledgment message – first only marking messages as sent, followed by marking them as consumed once the broker receives a specific acknowledgment message from the consumer. Unfortunately, this doesn't entirely solve the problem, and the consumer can still receive a message and fail to acknowledge its receipt. This approach also adds additional load on the system as it needs to keep track of multiple states for every single message.

Kafka overcomes this challenge by keeping track of the position of a consumer in each partition using a single integer which represents the offset of the next message to consume. This is possible since the Kafka log is totally ordered, and knowing the offset of the next message to consume can directly be inferred by this ordered log within each partition. The ordered log will be unique for each partition, which is consumed by exactly one consumer within each consumer group at any given time. This makes the 'acknowledgment message' very cheap computationally, and we can checkpoint these values periodically. This also shows us the benefit of Kafka's ordered log – being able to 'rewind' from the offset to re-consume data.

Keeping track of the log state is important to infer delivery semantics between the producer and consumer. Streaming applications can provide various message delivery guarantees:

At-most-once

Messages may be lost but will never be delivered more than once.

At-least-once

Messages will never be lost but may be delivered multiple times.

Exactly-once

The golden egg – messages are delivered once and only once.

These delivery guarantees can be considered from the perspective of both producers and consumers:

Producers:

For **producers**, Kafka provides a delivery guarantee somewhere between **at-least-once** and **exactly-once**, depending on how the system is configured. When a producer publishes a message to a log, the message is committed and will not be lost as long as a broker that replicates the partition to which the message is written does not fail.

But what happens if network disruptions stop the producer from receiving confirmation for a message which is successfully delivered? In this case, the producer may attempt to redeliver the same message, resulting in record duplication in our log. Fortunately for us, Kafka is resilient to such events and duplication when writing to the broker is prevented by assigning each producer a unique ID and tasking it to provide a sequence number for every message it sends. Under this setting, if a producer does resend data, the duplication can easily be detected by Kafka as the message will share a sequence number with a previous one and can simply be ignored.

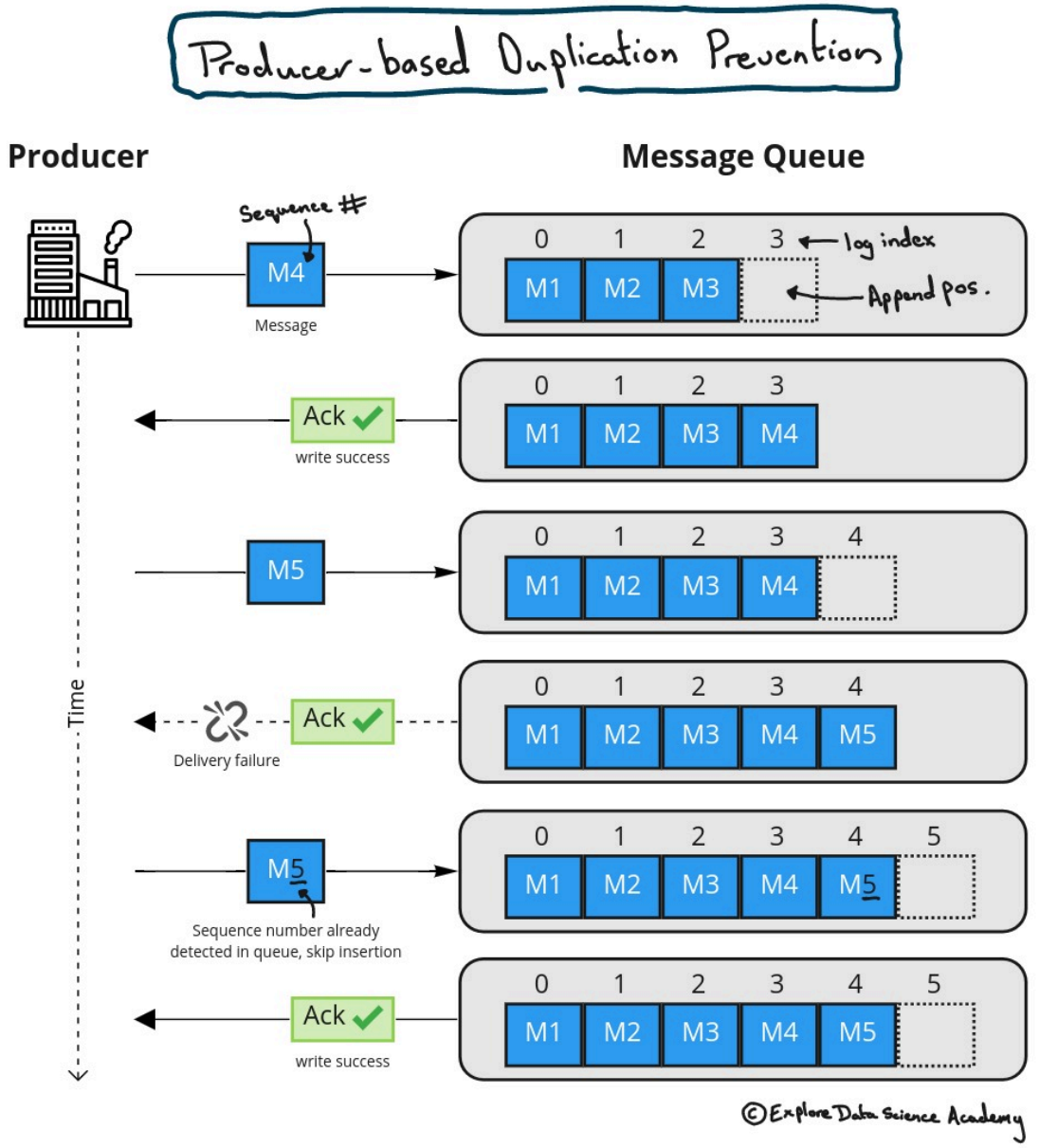


Figure 4: Kafka uses a simple message sequence number, provided by a producer for each message it publishes, to prevent duplicate record entries within its logs.

Consumers:

Delivery guarantees for **consumers** are more complex, unfortunately, as the relative read-position of a consumer within a log has to be tracked under the uncertainty of consumer or network failure. Here, two possible scenarios may arise as a consumer tracks its position within the log:

1. The consumer could read from the log, update its read-position, and then process the data. Under this setting, if a failure occurs during processing, then a new or revived consumer process will begin processing from the updated read-position, essentially losing the previously processed records which were not stored. This process correlates to an **at-most-once** delivery guarantee from Kafka to the consumer.
2. Alternatively, the consumer could read data from the log, process it, and then update its read-position. In this scenario, if a failure occurs during or after processing, before updating its position, then the new/revived consumer process will begin to process the same records again. This is consistent with an **at-least-once** delivery guarantee from Kafka to the consumer. Luckily, most events are associated with a unique identifier or primary key and so will not cause duplication errors downstream if reprocessing occurs.

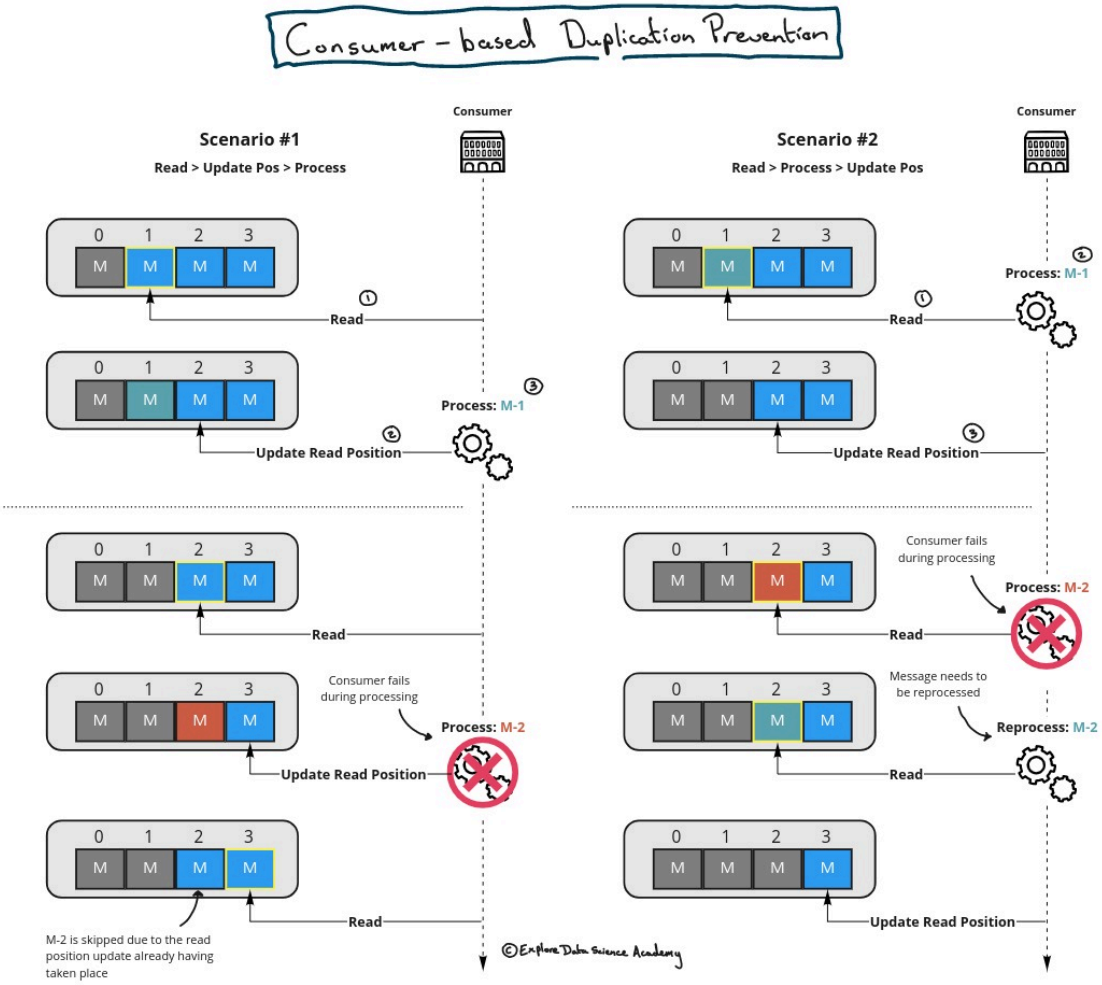


Figure 5: Depending on the sequence of steps taken by a consumer client when it reads from Kafka, message data are either skipped during consumer processing failure or data need to be reprocessed.

> It's important to note that Kafka also provides **exactly-once** delivery guarantees when performing internal routing between its own topics (known as Kafka Streams), where a Kafka topic is consumed and published to another topic) or when there is a high amount of coordination between Kafka and a consuming application.

For more information, refer to the Kafka documentation

Topics

Within Kafka, events are organised into topics. Each topic is a collection of records stored in a log-structured format. Topics thus serve as an organising mechanism and are analogous to folders within file systems, with a specific event being placed and indexed under a specific topic. Topics can have multiple producers and subscribers.

As is standard behavior for all message brokers, events in Kafka are not deleted from a topic once consumed and are retained until the defined lifetime for a message has passed. While retention time is set at the server level, it is also possible to configure retention time for specific topics.

Partitions

Topics are partitioned across Kafka brokers (described as sharding in the message broker section above). This partitioning allows for significant scalability because messages can be read from and written to in parallel from any of the partitions utilising the available Kafka message brokers. As mentioned before, messages are written to various brokers based on a key-index mechanism. Messages with similar keys will be written to the same broker, with the order of writing being preserved when reading messages. Partitioning allows for locality-sensitive processing in consumers, meaning that similar records or consecutive records will be located on the same partition, which can greatly speed up processing since less shuffling has to happen to get the correct records together to aggregate or sort on.

Below we illustrate how Kafka topics are partitioned and how this allows for parallel writing and reading of messages.

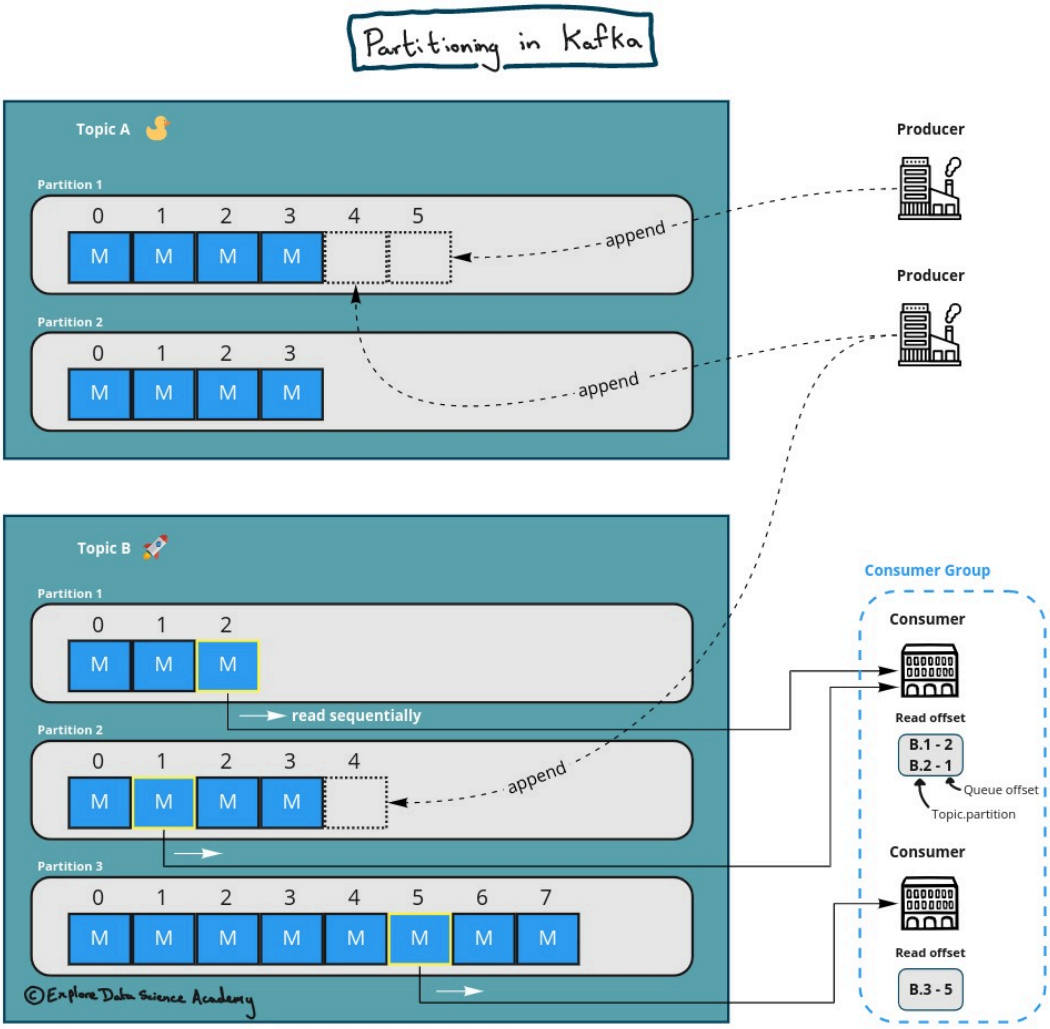


Figure 6: Multiple partitions allow for simultaneous reading and writing across topics for both consumers and producers.

Replication

Replication is central to Kafka, with all topic partitions being replicated by default. Each partition in Kafka has a single leader and zero or more followers. All reads and writes go to the leader of the partition, with the logs of the followers being identical to those on the leaders, having the same order and offsets. Once logs are written to the leader, the followers consume the logs in the same way as external consumer applications would. Using this strategy, replication enables fault-tolerance which can be realized within a local data center or across regions within the cloud.

The figure below depicts this process of replication, with leaders writing to two followers for two partitions within a single topic.

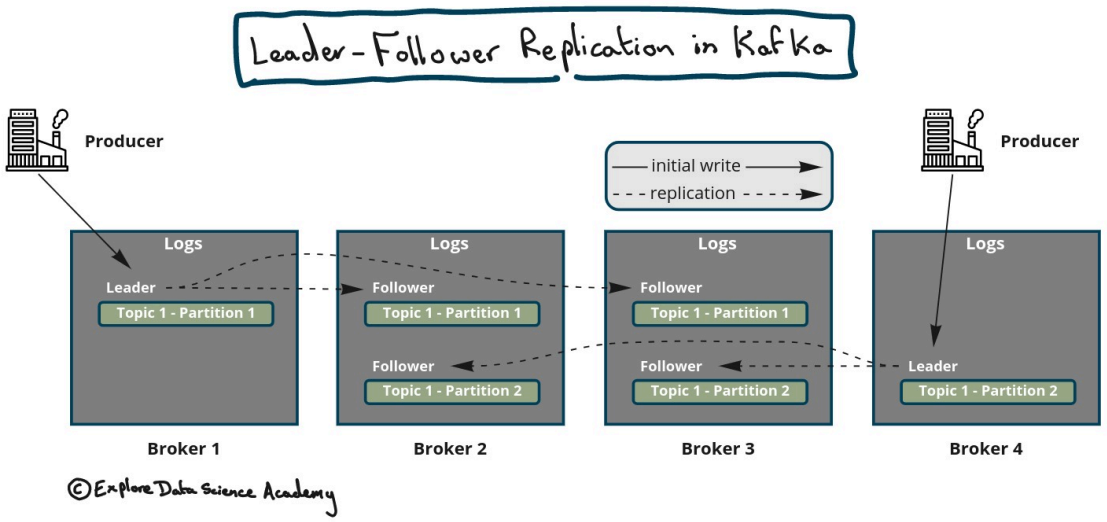


Figure 7: Once an event is written to a leader instance, it is subsequently consumed by follower instances.

Another feature of having a replicated system is that it gives us a concise definition of what a **committed message** is: a data record which is written to the logs of **all** replicas in our system. This then allows us to only give committed messages to consumers, providing them with guarantees that even if the leader or some of the replicas fail the consumer will still be able to receive the message.

Data Schemas

Schema definition is a central consideration when building a streaming application. This is because, until batch operations occur, streaming applications cannot assume a sufficient amount of data is available to infer the data schema. Here we have two options to help us deal with schema uncertainty:

- The first sees the use of bespoke data formats for streaming, such as Apache Avro, which supports an evolving schema that can be utilized across several consumers.
- The second approach to appropriately define a schema is accomplished by using data formats such as JSON, which inherently defines the data schema through the use of key-value pairs.

Apache Avro

Apache Avro is a row-oriented data serialization system. It relies on a JSON definition for data types and protocols and serialises the data into a compact binary format. Apache Avro needs schemas to structure the data that is being encoded.

Using Avro as a format to transfer data while streaming lets us take advantage of serialization, which is the process of turning complex objects into byte streams – transferring the data or objects in a platform-agnostic way – followed by deserialization at the other end.

The schema is defined using JSON:

```json { "type": "record", "name": "Drinks", "fields": [{"name": "ids", "type": ["int", "null"]}, {"name": "name", "type": "string"}, {"name": "size", "type": ["string", "null"]},] } ```

Schemas are composed of primitive types (**null**, **boolean**, **int**, **long**, **float**, **double**, **bytes**, and **string**) and complex types (**record**, **enum**, **array**, **map**, **union**, and **fixed**). Fields are defined via an array of objects which, in turn, each define a name and type.

Avro has APIs that support several prominent big data languages, including C++, Java, Python, and Scala.

While Avro has been the most dominant serialization method used alongside Kafka, other methods exist, including **JSON** and **Google Protocol Buffers (Protobuf)**. A comparison of these frameworks can be found here.

Conclusion

In this train, we've crossed the deep river of data streaming. Streaming and data streaming is a massive topic, and we can probably build a one-year course on the subject alone. The aim of this train, however, was to provide a theoretical overview of how a streaming application works. In our case, we used the popular open-source framework Apache Kafka to guide our learning.

While batch processing is still very prevalent in most industries and domains, we are increasingly moving to a place where streaming and seeing the world with a real-time perspective are becoming important to ensure competitiveness.

Key Takeaways
```

- **Real-Time Data Streaming:** Apache Kafka is fundamentally a message broker designed for high-throughput, real-time data streaming. It's used by major companies for applications requiring immediate data processing and analysis.
- **Distributed Architecture for Scalability and Fault Tolerance:** Kafka operates as a distributed system with brokers, producers, and consumers. This setup allows it to scale horizontally and maintain data integrity even if parts of the system fail.
- **Log-Based Storage with Topics and Partitions:** Kafka stores data in an ordered log format, organized into topics and partitions. This design ensures data persistence, ordering, and enables parallel processing for faster data handling.
- **Flexible Message Delivery Semantics:** Kafka offers different message delivery guarantees (at-most-once, at-least-once, exactly-once), allowing developers to choose the level of reliability needed for their applications.
- **Schema Management for Data Consistency:** Kafka supports schema definition using formats like Apache Avro, ensuring data consistency across producers and consumers. This is crucial for handling evolving data structures in streaming applications.